SDLC Model Comparison :-

(I) Classical Waterfall —

→ Basic
→ Rigid
→ Inflexible
→ Not for real project

(II) Iterative Waterfall :—

→ Basic
→ Problem is well understood.

(III) Prototype Model :—

→ User requirement not clear
→ Costly
→ No early lock on requirements
→ High user involvement.
→ Reusability

(IV) Incremental Model :

→ Module by module delivery
→ Easy to test and debug.

(V) Evolutionary Projects :—

→ large projects

(VI) RAD Model ⟶

→ Time and cost constraints

→ User at all levels

→ Reuseability

(VII) Spiral Model ⟶

→ Risk

→ Not for small projects

→ No early look on requirements

→ Less experience can work

(VIII) Agile ⟶

→ Flexible

→ Advanced

→ Parallel

→ Process divided into sprints.

Chapter 5: (Continue)

SRS: a document that is created when a detailed description of all aspects of the software to be built must be specified before the project is to commence.

Stakeholder: anyone who benefits in a direct or indirect way from the system which is being developed.

Eliciting Requirements:

(i) Meeting (ii) Rules (iii) Agenda
(iv) facilitator (v) definitio mechanism.

Quality function deployment has three types of requirement →

(i) Normal:

(ii) Expected

(iii) Exciting:

Common Elicitation Techniques:

(i) Interviews: one-on-one or group meetings to gather detailed insights.

(ii) Questionnaires/Surveys: Collect responses from a large number of people.

(iii) Observation/Job Shadowing: Watching users perform their tasks to discover hidden needs.

(iv) Brainstorming/Workshops: Group discussion to identify new ideas and system goals.

(v) Document Analysis: Studying existing manuals, reports and forms.

(vi) Prototyping: Creating mockups: or simple models to gather feedback.

(vii) Use cases | Scenarios: Describing typical user interactions step by step.

Techniques of Requirement Analysis:—

(i) Use case Modelling: Describes user-system interaction through "actors" and "use cases".

(ii) Context Diagram: Shows the system boundary and external entities interacting with it.

(iii) Data Flow Diagram: Represent data movement and processing withing the system.

(iv) E-R Diagram: Represent data object and their relationship.

(v) Class Diagram: (UML) Identifies system classes, attributes, and operations.

(vi) Behavioral Modeling: Describes system responses to events using state Diagrams.

(vii) Domain Modeling: Analyzes the real-world environment and its constraints.

Specification:

Contents of good SRS: →

(i) Introduction - P

(ii) Overall Description

(iii) Functional Requirement.

(iv) Non - functional Requirement.

(v) System Intefaces.

(vi) Validation Criteria.

Validation:

Correct: Meet actual user needs.

Complete: No missing information.

Consistent: No contradiction.

Feasible: Technically and economically possible.

Testable: Each requirement can be verified.

Types of Requirement: (SRS)*

(i) Functional requirement: Define specific system behavior of functions.
Ex: User can reset password through email.

(ii) Non-Functional requirement: Define performance or quality constraints.
Ex: "System should support 1000 users concurrently".

(iii) User Requirement: Natural language statements for end users.
Ex: "Doctors should be able to view patient's report".

(iv) System Requirement: Detailed technical requirements derived from user needs. Ex: System shall encrypt all patient data using AES-256."

(v) Domain Requirement: Derived from laws, policies or standards in the domain. Ex: "System must comply with medical data privacy law."

SRS: a formal document that describes in detail what the software system will do and how it will perform.

Characteristics:

Correct: Reflects the actual needs of the customer

Unambiguous: Every requirement has only one clear meaning.

Complete: All possible inputs, outputs and behaviors are covered.

Consistent: No conflicts among requirements.

Verifiable: Every requirement can be tested.

Feasible: Technically and economically achievable.

Modifiable: Easy update as requirements change.

Traceable: Each requirement can be linked to its origin and implementation.

Ith User Interfaces Design :- Laws, Rules, Issues and fixes.

UI : the bridge between the user and the software system.

Three golden rules of UI design —

(I) Place the user in control.

~~→ Users should~~

Principle —

(i) Provide undo/redo options.

(ii) Allow users to navigate freely.

(iii) Offer keyboard shortcuts for power users.

(iv) Provide meaningful defaults and let users customize settings.

(II) Reduce the User's Memory Load :

Principle —

(i) Keep actions visible and consistent.

(ii) Use menus, icons, and tooltips instead of commands

(iii) Group related information together.

(iv) Provide hints, example, and autofill suggestions

(III) Make the interface consistent:

Principle ——

(i) Consistent layout.

(ii) Use standard symbols.

(iii) Maintain consistent terminology across all screens.

(iv) Uniform error messages and help dialogs.

UI design Process ~~Interface~~:

1. User, Task and Environment Analysis
2. Interface Design Model.
3. Interface Construction.
4. Interface Validation.

Common Interface Design Issues and Fixes:-

1. Inconsistent Layouts
2. Poor Navigation
3. Overloaded Screens
4. Lack of Feedback

5. Unclear Labels or icons

6. Poor Error Messages

7. No Undo/Cancel Option

8. Accessibility Ignored

9. Slow Performance

10. Inconsistent Color or Fonts.

UI

Element:

1. Interface Layout.

→ Arrange elements logically

→ Keep primary actions in predictable areas.

→ Use grid alignment for structure.

2. Menus and Navigation:

→ Organize functions hierarchically

→ Keep menu names meaningful and consistent.

→ Highlight current page or active menu item.

3. Forms and Data Entry:

→ Use label – field alignment for clarity.

→ Provide inline validation.

→ Auto fill known data when possible.

4. Visual Design:

→ Maintain color harmony

→ Use contrast to highlight important element

→ Ensure readable font size and line spacing

~~→ feedback a~~

5. Feedback and Help:

→ Provide real time feedback for each action,

→ Include ~~tool~~tips, FAQ's. or help buttons.

→ Use loading indicators for longer operations.

UI Design Evaluation Methods:

(i) **Heuristic Evaluation:** Experts review UI using usability principles. (Early prototype stage)

(ii) **User Testing:** Real users perform tasks while observed. (Before final release).

(iii) **A/B Testing:** Compare two versions of a UI. (Optimization and post launch phase)

(iv) **Surveys / Feedback Forms:** Collect user satisfaction data. (After release)

A good interface:

→ Guides without confusing

→ Responds without delay.

→ Looks good without distraction

→ Empowers users to achieve goals effortlessly.

Prototype: an early working model of a system used to visualize and test the interface design before full development.

Types:

① Low-fidelity (Paper Prototype):

→ Simple sketches or wireframes.

Ex: paper screen drawings, whiteboard mockup.

② Midium fidelity: (Interactive Mockup):

: clickable demo without real data.

Ex: figma, Adobe xd.

③ High fidelity Prototype: Looks and behaves almost like the real system.

Ex: Web or App demo with real navigation.

benefits:

① Enables early user involvement and feedback

(ii) Detects usability problems before coding

(iii) Saves time and cost by avoiding rework later.

(iv) Clarifies requirements misunderstandings.

(v) Allows comparisons between design alterna

Ⓕ User feedback gathering process ⟶

design → test → refine → test →

Step 1: Define Evaluation Goals →

Step 2: Identify Target users.

Step 3: Select feedback techniques.

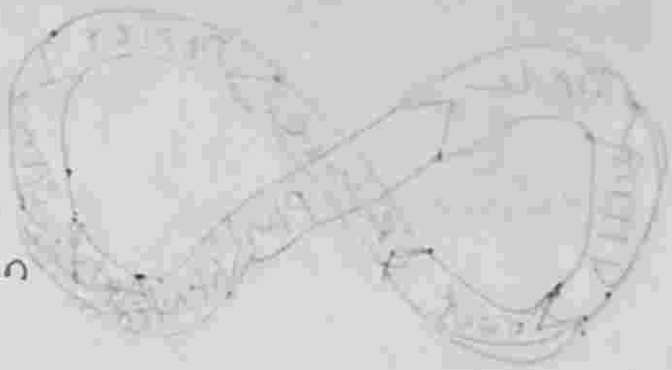
Step 4: Prepare Test environment.

Step 5: Conduct Feedback Sessions →

Step 6: Analyze and prioritize feedback

Step 7: Refine the design.

Techniques:



(i) Observation

(ii) Questionnaires and surveys

(iii) Interviews

(iv) Heuristic Evaluation

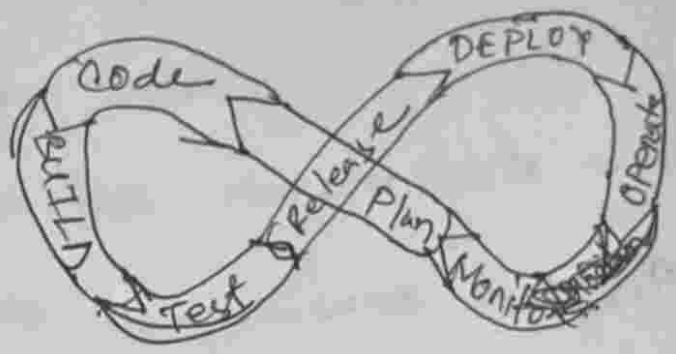
(v) A/B Testing

(vi) Usability Testing

(vii) Focus Group

(viii) Analytics and logs

DevOps:



Development: process of designing, coding, testing and building software applications or systems to meet user or business needs.

Main activities ——→

(I) Requirement Analysis

(II) Designing

(III) Coding

(IV) Testing

(V) Version Control.

Operations: to process, teams, and tools responsible for running, maintaining and manging software systems after they are developed, making sure they are available, reliable and secure.

Main responsibilities:

① Deploying software
② Monitoring Systems
③ Managing Infrastructure
④ Ensuring security and uptime
⑤ Handling incidents.

Core DevOps Practices:

1. **Continuous Integration (CI)**
   → Developers frequently merge their code into a shared repo.
   → Automated builds and tests run each

time to detects bug early.

Ex. Everty time a developers commit code, Jenkins runs unit tests. automatically

(2.) Continuous Delivery (CD)

→ Software is automatically built, tested and prepared for release to production with one click.

(3) Infrastructure as code: (IaC)

→ Managing servers and infrasture through code instead of manual setup.

Ex: Using Terraform or AWS Cloudformation to auto create servers and databases.

(4) Automated Testing:

→ Test run automatically to ensure reliability and avoid manual errors.

Ex: Selenium tests verify if a web page

works often each update.

(5.) Monitoring and Logging:

→ Continously tracking app performance and system health to detect issues early.

Ex: Tools like Prometheus or ELK Stack monitor CPU usage or app crashes.

(6.) Collaboration and Communication:

→ Developers, testers and operations work closely, often using shared tools like Slack, Jira or Github

Necessary:

→ Faster delivery: Automating build, test and deployment reduces time to market.

→ Better Quality: Continuous testing ensures fewer bugs.

Version Control Systems: software tool that records changes to files over time — so developers can

→ Revert to earlier versions.

→ Track who made changes and why.

→ Collaborate efficiently without overwriting.

Types —

① Local : keeps versions on one machine —

Ex: RCS (Revision Control System)

② Centralized : One central server stores all versions. Ex: CVS, Supervision.

③ Distributed : Every user has a full copy of repo.

Ex: Git, Mecurial.

Clone: Copy remote repo

Pull: Get updates from remote

Push: Upload local changes.

Commits: Save a snapshot.

Branch: Create a separate line of work.

Merge: Combine branches.

Merge conflicts: Occurs when two branches modify the same line or file region differently

Steps to resolve:—

1. Open the conflicting file(s)
2. Decide which version to keep
3. Delete the conflict markers.
4. Add and commit the resolved file.